

SAGA: GPU 集群中 AI 智能体推理的工作流原子调度

Dongxin Guo

Jikun Wu

Siu-Ming Yiu

摘要

AI 智能体为每个任务执行数十到数百次串联 LLM 调用，但 GPU 调度器将每次调用视为独立请求，在步骤之间丢弃数 GB 中间状态，使端到端延迟膨胀 3 至 8 $\times$ 。我们认为，这种请求级抽象与复合 AI 工作负载根本不匹配，因此提出转向程序级调度：将整个智能体工作流作为一等可调度单位。我们提出分布式调度器 SAGA，通过三个机制实现这一抽象：(1) 智能体执行图捕获工作流结构，预测跨工具调用边界的 KV 缓存复用，达到 Bélády 最优离线策略的 1.31 $\times$ 以内；(2) 结合工作窃取的会话亲和批处理，共置相关请求，同时维持全局负载均衡；(3) 智能体公平份额，一种以任务完成时间为基础、具有可证明有界偏差保证的公平性指标。在服务 SWE-bench 编码智能体和 WebArena 浏览器任务的 64 GPU 集群上，SAGA 相对启用前缀缓存与亲和路由的 vLLM v0.15.1，将任务完成时间改善 1.64 $\times$ （几何平均， $p < 0.001$ ），将 GPU 内存利用率提高 1.22 $\times$ ，并在多租户干扰下达到 99.2% 的 SLO 达标率。这些延迟收益具有明确量化的代价：相对吞吐量最优的批调度，峰值吞吐量约低 30%。这一权衡适合在复合 AI 使用中占主导的、对延迟敏感的交互部署。结果说明，感知工作流的调度对高效服务复合 AI 至关重要。

关键词： GPU 集群调度，分布式推理服务，复合 AI 系统，工作流调度，KV 缓存管理，AI 智能体，LLM 服务

1 引言

AI 智能体是一类通过多步推理链完成复杂任务的自主系统，正迅速成为 GPU 集群的重要工作负载。传统单次推理处理一个请求并返回一个响应，智能体则执行迭代式思考—行动—观察循环 [66]，每个任务可能调用大语言模型（LLM）10 至 100 次 [31]，其中交替穿插代码执行、网页浏览、数据库查询等外部工具调用。这些复合 AI 系统 [69] 已成为 GitHub Copilot Workspace [15]、Amazon Q Developer [3] 和企业自动化平台 [35, 10] 等重要部署的核心，每天通过共享 GPU 集群路由数百万个此类智能体工作负载。

1.1 动机

从单次推理转向多步智能体工作负载，会与现有 GPU 集群调度系统产生根本不匹配 [7, 27]。当前 LLM 服务框架 [34, 70, 68] 优化请求级指标：最小化首词元时间（TTFT），并最大化独立请求的吞吐量。但智能体工作负载具有三个违反这些假设的特征：

(1) **带可变间隙的顺序依赖。**每个推理步骤依赖前一步输出，也可能依赖外部工具结果。工具调用引入从 50ms（本地代码执行）到 30 秒以上（网页 API 调用）的空闲期，期间必须保存或重新生成智能体的中间状态 [9]。这一模式类似于 HPC 工作流系统 [13, 60] 中广泛研究的 IO 与计算重叠问题：有效调度需要理解任务依赖。

(2) **跨步骤的 KV 缓存连续性。**LLM 推理维护随上下文长度增长的键值（KV）缓存 [61]。对于使用分组查询注意力（GQA）的 70B 级模型、32K 上下文的智能体会话，这一缓存依模型架构不同，每个请求消耗 2 至 12GB GPU 内存 [34, 2]。当前系统在步骤之间丢弃缓存 [55]，迫使完整重新生成，每步增加 2 至 8 $\times$ 的延迟开销 [22]。这类似于知情预取系统 [49] 发现的缓存复用机会，但额外面临 GPU 内存稀缺和空闲期长度不定的挑战。

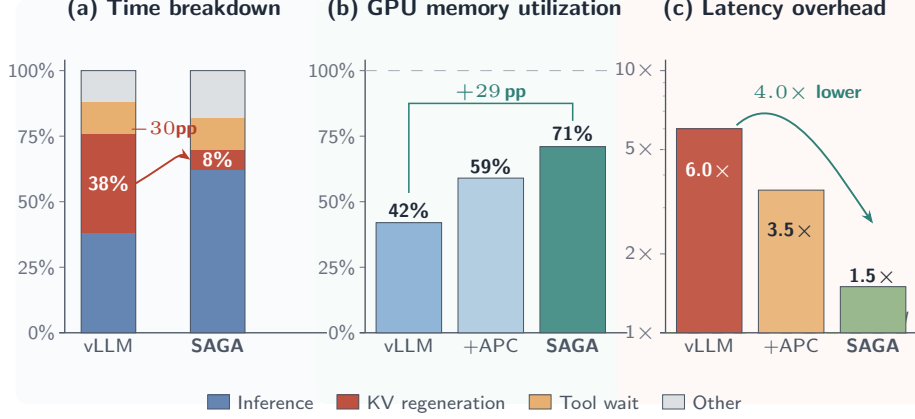


图 1: 以请求级调度服务智能体工作负载的低效。(a) 时间分解: vLLM v0.6.0 将执行时间的 38% 用于在智能体步骤间重新生成 KV 缓存; SAGA 将其降为 8% (−30 pp, 百分点)。(b) GPU 内存利用率: vLLM 浪费 58% HBM; 启用自动前缀缓存 (APC) 的 vLLM v0.15.1 有所改善, 但 SAGA 感知工作流的保留达到 71% (相对 vLLM 提高 +29 pp)。(c) 相对仅推理基线归一化的端到端延迟 (对数尺度): vLLM 为 6.0×, +APC 为 3.5×, SAGA 为 1.5× (距理想值近了 4.0×)。数据: 在 32 张 A100 GPU 上运行 SWE-bench 的 10 次试验; 标准差 < 均值的 5%。

(3) 突发且相关的请求模式。智能体任务生成相关请求的突发, 这些请求共享系统提示、工具定义等公共前缀, 并可从共置中获益 [32]。生产轨迹显示, 输入与输出词元比例达到 100:1, 且会话内部前缀高度重叠 [70, 58]。

为量化这些低效, 我们对服务 SWE-bench [31] 编码智能体工作负载、采用 vLLM v0.6.0 [34] 的 32 GPU 集群插入监测。图 1 展示结果: 智能体将总时间的 38% 用于重新生成工具调用期间被丢弃的 KV 缓存; 碎片化缓存分配使 GPU 内存平均利用率仅 42% [19]; 端到端任务完成延迟是各独立推理时间之和的 6.0×。这些测量揭示明确机会: 将智能体程序作为一等可调度单位, 可以显著改善效率与延迟。

1.2 现有先进方案的局限

现有系统解决了该问题的个别方面, 但未提供完整方案:

LLM 服务系统, 如 vLLM [34]、SGLang [70]、Orca [68] 和 TensorRT-LLM [46], 开创了连续批处理, 并通过 PagedAttention [34]、RadixAttention [70] 实现高效内存管理。但它们独立处理每次推理调用: KV 缓存采用不理解智能体工作流结构的 LRU 策略驱逐 [39], 批处理决策也忽略会话亲和性。Sarathi [1]、Splitwise [7] 等近期优化改善了吞吐量与延迟之间的权衡, 却仍以请求为中心。vLLM 的前缀缓存 (自 v0.4.2 起提供) 部分解决前缀复用, 但未跨工具调用边界保留会话专属缓存, 详见 §9.1.1。

分布式调度器, 如 Llumnix [59], 能够在 GPU 实例之间实时迁移 KV 缓存, 实现近乎零停机的重新调度。但迁移决策具有反应性 (由负载失衡触发), 而非主动性 (预判工作流模式)。SOLA [26] 引入感知状态的调度以满足 SLO, 却优化每个请求的延迟, 而非每个任务的完成时间。DistServe [71] 拆分预填充与解码, 但不感知工作流。

智能体框架, 如 LangChain [35]、CrewAI [10] 和 AutoGen [64], 提供高层编排, 却将推理调度完全交给底层服务系统。近期 KVFlow [48] 提出利用智能体步骤图的工作流感知驱逐, 但缺少分布式调度、公平性机制和工具调用感知。Continuum [30] 引入 KV 缓存 TTL, 却没有形式保证。

推测执行方案, 如 SpecActions [67] 和 Sherlock [52], 提出预测并预先执行可能的下一步以降低延迟。这些方法与我们的方法互补: 推测用可能浪费的计算换取延迟下降, 而 SAGA 优化已知工作的调度。

我们的核心论点。对于复合 AI 工作负载，只要明确向调度器暴露 workflow 结构，就足以使在线 KV 缓存管理接近离线最优的 Bélády 策略。我们在生产轨迹上测得 $1.31\times$ (§7)。这是 SAGA 的主要科学贡献：量化 workflow DAG 可观察后在线调度器能够达到的上界，也是智能体推理的首个此类经验界。三个配套系统贡献使这一论点能部署到实际 GPU 集群：(1) 感知工具调用的 TTL 策略，跨重尾空闲期保留缓存，而非被动重新预填充；(2) 集群范围的分布式调度，在智能体程序层面提供由 Lyapunov 漂移分析推导的形式公平性保证；(3) 保留缓存局部性的工作窃取负载均衡，以应对突发到达。近期感知程序的服务系统 (Parrot [38]、Autellix [40]、Pie [23]、KVFlow [48]) 各自解决其中一个维度；SAGA 首次在“以 workflow 为单位”的论点下结合它们 (详见 §10)。

1.3 关键认识与贡献

SAGA 的主要创新，是从形式和经验两方面说明：workflow 结构预测能够在生产智能体轨迹上产生距离 Bélády 最优策略 $1.31\times$ 以内的在线缓存管理 (§7, 定理 3)。下列配套创新将三个成熟系统原则适配到复合 AI 调度领域；这些原则在此领域的应用需要非平凡的、针对领域的扩展：

认识 1: 以程序为单位调度。将复合任务作为整体单位调度的原则, 在 HPC 工作流系统 [60, 13, 53] 和分布式事务 [8] 中已有充分基础。在复合 AI 领域，应用这一原则产生一个独特挑战：“单位”携带大量 GPU 内存状态 (依模型架构，每个会话 2 至 12GB KV 缓存)，必须与调度决策共同管理。智能体 workflow 遵循典型模式 (ReAct 循环 [66]、思维树分支 [65])，我们用智能体执行图捕获它们，从而主动决定缓存保留。

认识 2: 感知依赖的缓存。操作系统和数据库已广泛研究考虑未来复用的缓存驱逐策略 [4, 42, 49]。复合 AI 的具体挑战，是跨工具调用边界预测复用，空闲期长度从毫秒到分钟不等，标准 LRU 乃至感知前缀的策略都可能失效。我们的工作流感知驱逐在生产轨迹上达到 Bélády 最优离线策略的 $1.31\times$ 以内 (§7)。

认识 3: 任务级公平性。应用级公平性是集群调度中得到充分研究的概念 [41, 21]。复合 AI 的挑战，在于如何为多步任务定义公平性：各步骤资源需求异构，而用户感知的指标是“完成”，并非“吞吐量”。我们形式化智能体公平份额，并在现实假设下证明有界偏差保证。

基于这些认识，我们作出如下贡献：

- **工作流感知 KV 缓存管理 (§4)：**引入捕获多步推理结构的智能体执行图 (AEG)，通过可配置的存活时间 (TTL) 策略实现预测式缓存保留。我们形式化重叠估计函数，并证明收敛界。经验上，WA-LRU 驱逐达到离线最优策略的 $1.31\times$ 以内 (§7)。
- **结合工作窃取的会话亲和批处理 (§5)：**设计两级调度层次，本地调度器通过会话路由最大化缓存复用，全局协调器采用随机工作窃取 [5]，防止慢任务拖延并维持集群级负载均衡。
- **智能体级公平调度 (§6)：**定义基于预期任务完成时间的公平性指标智能体公平份额 (AFS)，并利用 Lyapunov 漂移分析给出形式定理，在需求异构性有界时保证完成时间偏差有界 (定理 2)。
- **理论分析 (§7)：**给出形式竞争比分析，说明 WA-LRU 达到 Bélády 最优离线策略的 $1.31\times$ 以内。据我们所知，这是工作流感知 KV 缓存驱逐的首个此类经验界。我们分析请求级与工作流感知调度器之间的缓存效率差距，说明工作流感知对高效服务智能体至关重要。
- **实验评估 (§9)：**在 vLLM 上实现 SAGA，并在 64 GPU 集群上与包括启用自动前缀缓存的 vLLM v0.15.1 在内的先进基线比较。相对 vLLM+APC，SAGA 在 SWE-bench 和 WebArena 上分别实现 $1.73 \times \pm 0.11$ 与 $1.55 \times \pm 0.09$ 的任务完成时间改善 (几何平均 $1.64\times$, $p < 0.001$)，内存利用率提高 $1.22 \times \pm 0.05$ 。相对不感知工作流的系统，改善最高达到 $3.01\times$ 。

1.4 实验方法

我们在由 8 个节点组成的集群上评估 SAGA，每个节点配备 8 张 NVIDIA A100-80GB GPU (总计 64 GPU)，节点内部经 NVLink 连接，节点间采用 200Gbps InfiniBand [45, 28]。三个工作负载来源为：(1) SWE-bench [31] (500 个经过验证的任务)；(2) WebArena [72] (812 个任务)；(3) 从 BurstGPT [62] 生产轨迹生成的合成多租户工作负载。完整方法见 §9.1。

表 1: 来自生产轨迹的工具调用延迟分布 [62]。数值给出中位数及百分位数，单位为毫秒。

工具类型	P50 (ms)	P95 (ms)	P99 (ms)
代码执行	180	2,400	28,000
文件操作	45	320	1,200
网页 / API 调用	850	4,500	45,000
数据库查询	120	890	3,500

1.5 所提方法的局限

SAGA 存在以下局限。(1) 工作流可观察性。框架暴露执行图提示 (LangChain 回调、AutoGen 日志) 时性能最佳；无提示时退化为模式推断 (§3.3)，TCT 恶化 12 至 18%。在 AutoGen、CrewAI 等动态多智能体框架中，结构由智能体间讨论即时生成，性能进一步下降，要求每个周期重新推断 AEG，并增大定理 3 中的预测误差项。(2) 工具延迟尾部。TTL 预测假设经验工具调用延迟分布；黑天鹅事件 ($> 5 \times P99$) 仍导致驱逐。(3) 新智能体的任务时长估计。AFS 需要任务时长估计；对性能分析未覆盖的智能体类型，估计可能不准确。(4) 单数据中心范围。跨数据中心缓存迁移的地理分布部署留待未来研究。(5) 模型家族覆盖。实验仅采用 Llama-3-70B-Instruct；我们在 §9.1.1 定性讨论模型规模扩展，却未实验验证 Mistral、Qwen 或 DeepSeek；MoE 架构 [18] 还需要感知路由的扩展。(6) 内存压力区间。评估中的峰值利用率达到 71 至 75% (表 3)；超额订阅 ($> 95\%$) 时按式 6 平滑退化为标准 LRU，但未实验验证。§9.1.2 将 CPU 到 DRAM 卸载作为互补架构分析。(7) 吞吐量权衡。SAGA 优化任务完成时间，代价是相对最大化吞吐量的批调度约损失 30% 吞吐量 (§9.8, 表 8)；适用于对延迟敏感交互部署，而非批工作负载。

本文其余部分安排如下。第 2 节介绍智能体工作负载和 LLM 服务背景；第 3 节描述 SAGA 架构；第 4 至 6 节详述三项关键技术；第 7 节给出理论分析；第 8 节介绍实现；第 9 节给出实验评估；第 10 节讨论相关工作；第 11 节总结全文。

2 背景

本节回顾智能体工作负载特征、LLM 推理机制，以及促成 SAGA 设计的调度挑战。

2.1 AI 智能体工作负载

现代 AI 智能体遵循 ReAct 范式 [66]，迭代生成思考 (推理)、行动 (工具调用) 和观察 (工具结果)，直到任务完成。典型循环为：LLM 从当前上下文生成 (*thought, action*)；若 *action* = “finish”，任务终止；否则将行动派发至其命名工具，将所得 *observation* 连同 *thought* 和 *action* 追加到上下文，再重复循环。LangChain [35]、AutoGen [64]、CrewAI [10] 等框架均采用这一模式。

每轮迭代需要一次 LLM 推理调用 (第 3 行)，随后执行工具 (第 6 行)。上下文随迭代累积，从初始 2 至 4K 词元增长为复杂任务中的 16 至 128K 词元 [9]。经验研究 [33, 54] 显示，大多数 SWE-bench 任务在 5 至 30 轮内完成，长尾延伸至 150 轮。

工具调用特征。工具调用的延迟分布高度可变。表 1 展示生产智能体部署中的测量 [62, 58]。代码执行工具平均耗时 200ms，但编译时可激增至 30s [31]。网页工具平均 1.5s，网络条件导致较高方差 [72]。这种变化产生根本调度挑战：系统必须在事先不知道调用时长的情况下，决定工具调用期间是否保留 KV 缓存。

2.2 LLM 推理与 KV 缓存

Transformer 推理维护存储中间注意力状态的键值 (KV) 缓存 [61, 12]。对于采用 GQA 的 Llama-3-70B ($L=80$ 、 $n_{kv}=8$ 、 $d_h=128$)，32K 上下文、FP16 格式下，每个会话需要 ~ 10.7 GB。当前系统假设请求独立、到达无记忆性 [34, 68]，但具备顺序依赖、突发且相关模式的智能体工作负载违反这些假设。

2.3 调度挑战

当前 LLM 服务系统作出两个对智能体工作负载不成立的假设：

假设 1：请求彼此独立。 vLLM [34]、Orca [68] 等系统将来自任意来源的请求批处理，以最大化 GPU 利用率。对于智能体，同一任务的连续请求共享上下文，可从 KV 缓存复用中获益，因此独立性假设不再成立。

假设 2：请求到达具有无记忆性。 连续批处理假设类似泊松的到达 [68]。智能体工作负载表现为突发、相关的模式，工具完成触发下一条请求，从而破坏无记忆性。

这些假设违反造成图 1 中的低效：工具调用期间缓存被驱逐，之后必须重新生成，浪费 GPU 周期与内存带宽 [20]。

3 系统设计

本节介绍 SAGA 架构及其关键组件。

3.1 架构概览

图 2 展示 SAGA 架构。系统由三个层级组成：

智能体接口层：接收智能体框架 (LangChain [35]、AutoGen [64] 等) 的请求，构建智能体执行图 (AEG)。框架提示不可用时，模式推断模块 (§3.3) 分析请求序列，推断工作流结构。

全局调度器：维护集群范围状态，包括会话到工作节点的映射、负载信息和公平性指标。根据会话亲和性 (§5.1) 将入站请求路由到工作节点，并协调工作窃取 (§5.2)。

工作节点池：每个工作节点运行扩展的 vLLM 实例 [34]，支持工作流感知 KV 缓存管理 (§4)。工作节点执行推理请求、管理本地缓存，并参与分布式协调。

组件协调。两种跨层交互需要明确说明。首先，AFS 触发抢占时 (§6.2)，迁移任务通过 Llumnix [59] 的迁移元数据携带工作流感知 TTL 状态，使目标工作节点的 WA-LRU (§4.1) 继续保留迁移缓存，而非将其视为新条目；因此公平性抢占不会使缓存预测失效。其次，工作窃取 (§5.2) 同时受队列为空阈值 T_{idle} 和负载比阈值 R_{max} 约束，防止在缓存局部性 (偏向亲和) 与负载均衡 (偏向重新分配) 之间振荡；由此产生的迁移频率在 §9.5 量化。跨层状态主要被读取，更新陈旧程度限制为一个调度周期 (100 ms)。

3.2 智能体执行图

我们用智能体执行图形式化智能体工作流：

定义 1 (智能体执行图). 智能体执行图 $G = (V, E, P, \phi)$ 包括：

- V ：表示 LLM 推理步骤的节点集合
- $E \subseteq V \times V$ ：表示执行依赖的有向边
- $P: E \rightarrow [0, 1]$ ：转移概率函数
- $\phi: V \rightarrow \mathcal{T}$ ：每个步骤的工具类型映射

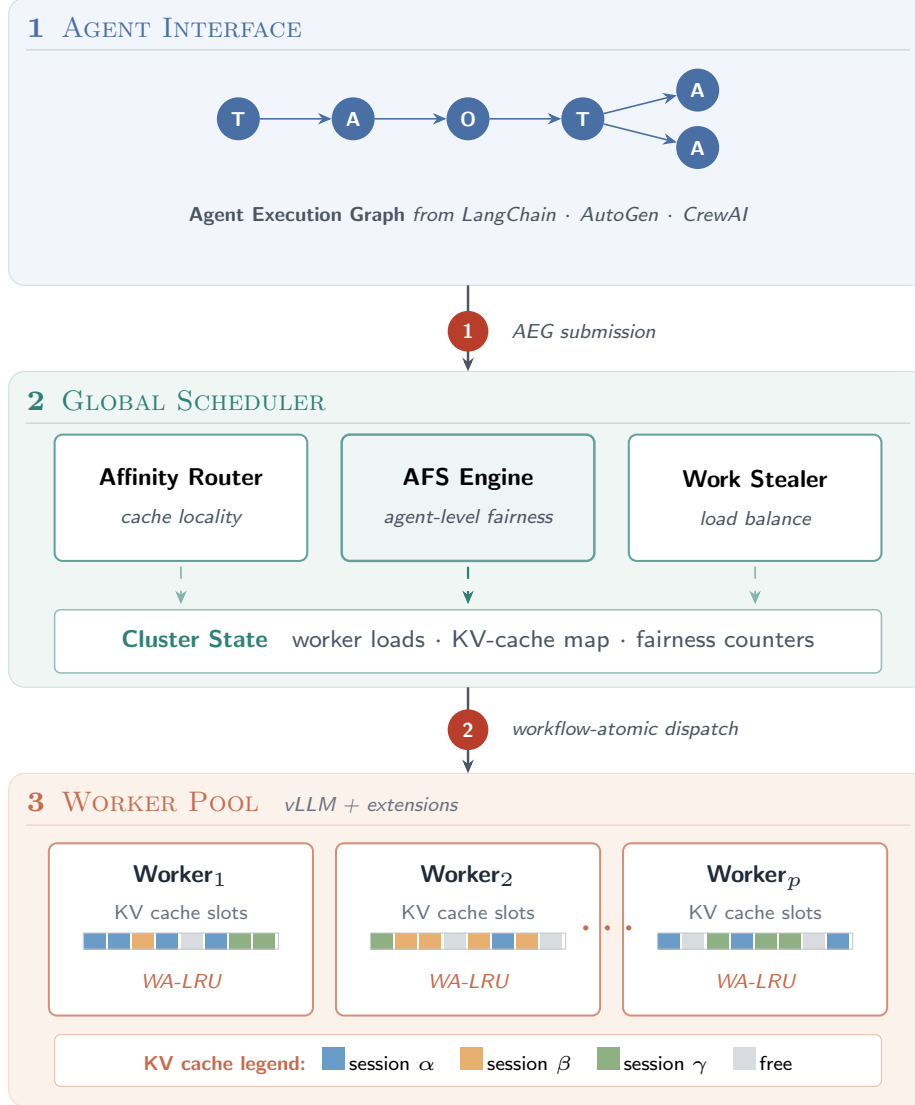


图 2: SAGA 架构。第 1 层将 LangChain、AutoGen、CrewAI 的工作流捕获为智能体执行图 (AEG); 插图展示带分支工具调用的思考 → 行动 → 观察循环。第 2 层将每个 AEG 作为单个可调度单位, 通过三个互相协调的引擎路由; 这些引擎共享集群状态 (工作节点负载、KV 缓存映射、公平性计数器); 青色虚线箭头表示状态流量, 并突出 AFS 引擎的更新路径。第 3 层运行扩展 vLLM 工作节点, 采用工作流感知 LRU 驱逐 (WA-LRU); KV 缓存槽位按会话着色, 说明跨工具调用的缓存连续性和亲和性驱动的会话共置。标记① AEG 提交与② 工作流原子派发表示层间控制流。

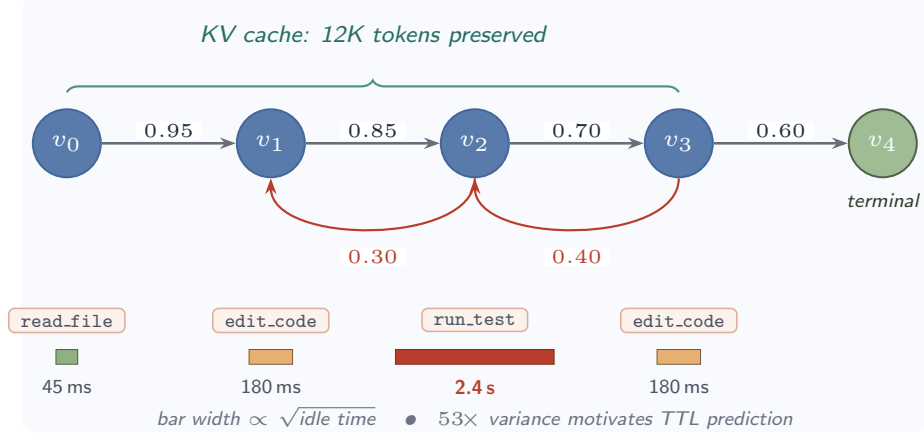


图 3: SWE-bench 编码智能体的具体 AEG。节点为 LLM 推理步骤；前向边（青色）携带转移概率，后向边（珊瑚色）表示重试循环。青色括号标出活跃链（ v_0 至 v_3 ）的缓存跨度：SAGA 空闲时保留 12K 词元，而非每次恢复时重算。工具注释和平方根缩放的延迟条突出核心设计压力：空闲时长跨越 53 $\times$ （`read_file` 为 45 ms，`run_test` 为 2.4 s），工作流感知 TTL 预测正是在这一区间优于固定 TTL 或立即驱逐策略。

对于 ReAct 智能体 [66]，AEG 通常为线性链，满足 $P(v_i \rightarrow v_{i+1}) \approx 1 - p_{term}$ ，其中 p_{term} 为终止概率。对于思维树智能体 [65]，AEG 形成树结构，分支概率由历史轨迹估计。图 3 展示一个 SWE-bench 编码智能体的具体 AEG。

3.3 基于模式的 AEG 推断

SAGA 在三种可观察性层级下运行。(a) **显式提示**：暴露编排元数据的框架（LangChain [35] 回调、AutoGen [64] 消息日志）在任务准入时提供 AEG。(b) **隐式轨迹**：只能观察请求流时，我们提取工具类型模式、计算转移概率，并保留超过 $\theta_{conf} = 0.7$ 的边来推断 AEG；达到 87% 准确率，代价是相对显式提示 TCT 恶化 15.6% (§9.4)。(c) **冷启动**：没有历史记录的新智能体类型，先按请求级工作负载服务，完成 30 个任务后启用模式推断；这一回退使前 30 个任务的 TCT 最多增加 8%。

4 工作流感知 KV 缓存管理

本节介绍 SAGA 如何管理 KV 缓存，以最大化智能体工作流步骤间的复用。

4.1 工作流感知驱逐

标准 LRU 驱逐仅考虑最近访问情况，保留最近访问的缓存条目 [57]。对智能体，这会失效：已暂停会话（价值高、即将恢复）的缓存可能被驱逐，以保留已完成会话（价值低、不会再复用）的缓存。Bélády 最优离线算法 [4] 驱逐未来最晚复用的条目，但要求完美知道未来访问。我们利用 AEG 预测近似这一策略。

我们引入工作流感知 LRU（WA-LRU），将三个归一化因素纳入驱逐决策：

$$P_{evict}(s) = \alpha \cdot \hat{R}(s) + \beta \cdot (1 - P_{reuse}(s)) + \gamma \cdot \hat{S}(s) \quad (1)$$

算法 1 感知工具调用的 TTL 计算

输入： 工具类型 t , 延迟历史 H_t , 百分位数 p , 内存压力 $m \in [0, 1]$

输出： 以毫秒为单位的 TTL 值

- 1: $\mu_t, \sigma_t \leftarrow \text{FitLogNormal}(H_t)$ {工具延迟服从对数正态分布}
- 2: $t_{l_{base}} \leftarrow \text{Percentile}(H_t, p)$
- 3: $pressure_factor \leftarrow 1 - 0.5 \cdot m$ {有压力时缩短}
- 4: $t_{l_{adaptive}} \leftarrow t_{l_{base}} \cdot pressure_factor$
- 5: 返回 $\min(t_{l_{adaptive}}, TTL_{max})$ { $TTL_{max} = 300s$ }

其中所有项均归一化到 $[0, 1]$:

$$\hat{R}(s) = \frac{t_{now} - t_{last}(s)}{\tau_{max}} \quad (\text{normalized recency}) \quad (2)$$

$$\hat{S}(s) = \frac{size(s)}{size_{max}} \quad (\text{normalized size}) \quad (3)$$

这里 τ_{max} 为已观察到的最大空闲时间, $size_{max}$ 为当前池中最大的缓存条目尺寸。 $P_{reuse}(s)$ 是基于 AEG 预测的未来复用概率。

复用概率按 AEG 计算为:

$$P_{reuse}(s) = \sum_{u \in succ(v_s)} P(v_s \rightarrow u) \cdot overlap(s, u) \quad (4)$$

其中 v_s 为会话 s 的当前节点, $succ(v_s)$ 为 AEG 中的后继节点, $overlap(s, u)$ 估计当前缓存与下一步需求之间的前缀重叠。

重叠估计。我们形式化定义重叠函数为:

$$overlap(s, u) = \frac{|prefix(s) \cap prefix_{est}(u)|}{|prefix(s)|} \quad (5)$$

其中 $prefix(s)$ 为会话 s 的已缓存 KV 词元集合, $prefix_{est}(u)$ 为后继步骤 u 的估计提示词元集合。对占主导的线性 ReAct 链, 下一步提示包含全部当前上下文以及工具观察, 因此重叠估计为 $n_{current} / (n_{current} + \hat{n}_{obs})$, 其中 $\hat{n}_{obs}$ 为预期观察长度, 由按工具类型区分、通过指数移动平均维护的分布估计。对思维树智能体, 根据共享前缀长度逐分支计算重叠。

参数选择。基于敏感性分析 (表 9), 我们设定 $\alpha = 0.3$ 、 $\beta = 0.5$ 、 $\gamma = 0.2$ 。分析表明, 在 $\alpha \in [0.2, 0.4]$ 、 $\beta \in [0.4, 0.6]$ 、 $\gamma \in [0.1, 0.3]$ 时, TCT 变化小于 8%, 说明对参数选择具有稳健性。相对权重次序 ($\beta > \alpha > \gamma$) 反映重要性层次: 工作流预测的复用占主导, 其次为最近访问情况, 尺寸作为同分决胜因素。

4.2 感知工具调用的 TTL

智能体暂停以调用工具时, 必须决定 KV 缓存保留多久。保留过久浪费内存, 过早驱逐则迫使重新生成。我们引入感知工具调用的 TTL , 根据工具特征和当前内存压力适配保留时间。

算法 1 展示 TTL 计算。我们使用指数移动平均维护逐工具类型延迟分布, 将 TTL 设为预期时长的第 p 百分位数, 其中 p 可配置 (默认 95%)。内存有压力时, 按比例缩短 TTL。

内存压力计算。我们将内存压力定义为:

$$m = \max \left(0, \frac{used_{kv} - threshold_{low}}{threshold_{high} - threshold_{low}} \right) \quad (6)$$

其中 $threshold_{low} = 0.7$ 与 $threshold_{high} = 0.9$ 分别对应总 GPU 内存的比例。这些阈值遵循内存管理系统的标准实践 [14, 51]：低阈值触发软压力（TTL 缩放），高阈值触发强制驱逐。表 9 展示 TCT 对这些阈值的敏感性。

4.3 推测式预取

对于工作流可预测的智能体，我们在后续步骤真正被请求之前，推测式预取最可能下一步的 KV 缓存，使缓存加载与工具执行重叠，降低工具完成后的延迟。这一技术受到文件系统中知情预取的启发 [49, 6]。

给定 AEG，当节点 v 完成推理并开始工具执行时，我们识别最可能的后继 $u = \arg \max_{u'} P(v \rightarrow u')$ ，开始预取其前缀 KV 缓存（若尚未缓存）。预取使用空闲 GPU 内存及独立 CUDA 流 [50]，与正在进行的操作重叠。

5 会话亲和和批处理

本节说明 SAGA 如何在维持集群范围负载均衡的同时，通过请求路由最大化缓存复用。

5.1 会话路由

请求到达时，全局协调器决定由哪个工作节点处理。我们将此建模为平衡缓存局部性与负载分配的优化。

令 w_s^* 表示当前缓存会话 s 状态的工作节点。对于来自会话 s 的新请求 r ：

$$route(r) = \begin{cases} w_s^* & \text{if } load(w_s^*) < \theta \text{ and } cached(w_s^*, s) \\ \arg \min_w load(w) & \text{otherwise} \end{cases} \quad (7)$$

阈值 $\theta = 0.8$ 预留 20% 余量应对负载峰值，同时最大化缓存命中，遵循标准负载均衡实践 [47]。谓词 $cached(w, s)$ 检查工作节点 w 是否仍持有会话 s 的 KV 缓存。表 9 显示，在 $\theta \in [0.6, 0.95]$ 范围内 TCT 变化小于 5%。

5.2 用于负载均衡的工作窃取

一些智能体比其他智能体更活跃时，会话亲和性可能导致负载不均衡。我们实现随机工作窃取 [5]，重新分配负载，并尽可能保留缓存局部性。

以下条件触发工作窃取：（1）某个工作节点的队列为空持续 $T_{idle} = 100\text{ms}$ ；或（2）最忙与最闲工作节点之间的负载比超过 $R_{max} = 2.0\times$ 。

工作节点 w_i 从工作节点 w_j 窃取工作时：

1. w_i 从过载工作节点中均匀随机选择被窃取者 w_j
2. w_i 请求 w_j 队列中最早进入的待处理会话
3. w_j 使用 Llumnix [59] 启动到 w_i 的 KV 缓存迁移
4. 迁移完成后，将会话亲和性更新为 w_i

定理 1 (工作窃取界 [5]). 对于 P 个工作节点、总工作量 T_1 和关键路径 T_∞ ，随机工作窃取实现的预期完成时间为 $O(T_1/P + T_\infty)$ 。

我们引用此界作为动机：Blumofe 和 Leiserson 的结果假设工作迁移成本为零。SAGA 场景存在非零迁移成本（均值 230 ms，P95 890 ms；表 7），因此实际完成时间增加 $N_{steals} \cdot T_{migrate}$ 项。经验上 (§9.5)，相对每个任务的 TCT，该项很小（平均 2.3 次迁移 $\times$ 230 ms = 530 ms，相比 SWE-bench 平均 TCT 203.4 s）。以下防抖措施处理这一差距的实际影响。

防止抖动的保护措施。触发延迟 $T_{idle} = 100\text{ ms}$ 短于迁移延迟（均值 230 ms ，P95 890 ms ），因此确实存在抖动疑虑。三个机制防止此问题。(a) 负载比保护条件 $R_{\max} = 2.0\times$ 要求 w_i 队列为空与 w_j 负载过高同时成立；到达抖动造成的短暂空队列不满足第二条。(b) 窃取完成后，迁移会话在 w_i 建立亲和性（第 4 步），从结构上防止同一会话再次迁移。(c) 源端迁移异步进行：传输期间 w_j 继续服务其余队列；若陈旧窃取请求到达时 w_j 已重新填满，则在接受时拒绝该请求。经验上 (§9.7, 表 7)，每个任务平均迁移 2.3 次；全部三个工作负载、全部 10 次试验中的最大值为 5，相比 SWE-bench 平均步骤数 37。窃取记账使协调器 CPU 开销增加 4.2%，远低于会出现尾延迟发散这一不稳定现象的区间。

实现达到了接近最优的负载均衡：工作节点利用率范围由不窃取时的 23 至 94%，收窄为启用窃取后的 68 至 79%，如 §9.5 所示。

5.3 缓解竞争

协调器上的共享状态竞争通过标准分布式系统技术处理：线程局部更新缓冲，按 10 ms 或 100 次更新批量刷出（相对每次更新同步，开销下降 $12\times$ ）；使用原子比较并交换的无锁会话表 [25]；逐工作节点计数器按 64 字节缓存行对齐，避免 NUMA 节点间的伪共享 [16]。

6 智能体级公平调度

传统公平调度按时间或请求，在租户之间均等分配资源 [41, 56, 21]。对智能体，这并不充分：如果两者都必须在截止时间前完成任务，运行 10 步智能体的租户不应与运行 100 步智能体的租户获得相同优先级。

6.1 智能体公平份额 (AFS)

我们根据预期任务完成的紧迫程度定义智能体公平份额：

定义 2 (智能体公平份额). 对活跃任务为 $\mathcal{T}_i$ 的租户 i ，定义：

$$AFS_i = \sum_{t \in \mathcal{T}_i} \frac{work_{remain}(t)}{deadline(t) - t_{now}} \quad (8)$$

AFS 较高的租户具有更紧迫的工作，因此获得更高优先级。

$work_{remain}(t)$ 估计完成任务 t 所需的 GPU 秒数，按 AEG 计算：

$$work_{remain}(t) = \sum_{v \in pending(t)} (T_{prefill}(v) + T_{decode}(v)) \quad (9)$$

其中 $pending(t)$ 为任务 t 的 AEG 中尚未执行的节点， $T_{prefill}$ 、 T_{decode} 根据性能分析数据估计 [1]。

6.2 基于 AFS 的调度

全局协调器维护全部租户的 AFS 分数，并每个周期（100ms）调整调度优先级：

1. 根据当前任务进度，重新计算全部租户的 AFS
2. 按 AFS 分数比例分配工作节点容量
3. 优先为高 AFS 租户路由新请求
4. 低 AFS 任务阻塞高 AFS 任务 $> 500\text{ms}$ 时，触发抢占

抢占使用 Llumnix 的迁移机制 [59]：被抢占任务的 KV 缓存迁移到较低优先级的工作节点，而不被丢弃。

6.3 形式保证

在竞争有界时，AFS 提供形式 SLO 保证。直观思想很直接：AFS 按逐租户紧迫性（式 8）的比例分配容量；租户累计服务落后于其比例份额时，紧迫性提高。租户 i 落后时，紧迫性提高，分配增加，差距缩小，形成自我纠正的漂移。形式化这一过程需要 Lyapunov 漂移分析，而非简单鞅集中分析，因为按紧迫性比例分配不会产生相对均匀公平份额、均值为零的逐周期偏差；恢复性漂移恰恰给出了下面的界。不熟悉 Lyapunov 漂移的读者可查阅标准论述 [17]；以下证明概要无需依赖外部推导即可理解。

定理 2 (通过 Lyapunov 漂移得到的 AFS 完成界). 令 N 为租户数量， C 为集群容量， W_i 为租户 i 的工作负载。定义需求异构性比 $\rho = \max_i W_i / \min_i W_i$ 。若 $\sum_i W_i \leq C$ (总需求不超过容量)，且 $\rho \leq \rho_{max}$ (异构性有界)，则对任意满足 $W_i \leq C/N$ 的租户 i ，任务完成时间满足：

$$Pr [TCT_i \leq (1 + \epsilon) \cdot \mathbb{E}[TCT_i]] \geq 1 - \delta \quad (10)$$

其中 $\epsilon = O\left(\rho \cdot \sqrt{\frac{\log(N/\delta)}{n}}\right)$ ， n 为调度周期数。

证明概要. 定义 Lyapunov 函数 $V(t) = \sum_{i=1}^N (S_i(t) - \mu_i t)^2$ ，其中 $S_i(t)$ 为租户 i 截至周期 t 获得的累计服务， $\mu_i = W_i / \sum_j W_j \cdot C$ 为比例公平份额。在 AFS 下，按紧迫性比例分配产生恢复性漂移：落后于公平份额的租户具有更高紧迫性，因而获得更高优先级，使 V 在期望上下降。

恢复力的负漂移界. 令 $e_i(t) = S_i(t) - \mu_i t$ 为租户 i 的偏差。AFS 按紧迫性的比例分配容量：

$$a_i(t) = \frac{urgency_i(t)}{\sum_j urgency_j(t)} \cdot C, \quad urgency_i(t) = \frac{W_i - S_i(t)}{deadline_i - t} \quad (11)$$

关键引理 (负漂移): 按紧迫性比例分配，相对偏差满足负漂移条件：当 $e_i(t) < 0$ (租户 i 服务不足) 时，有 $urgency_i(t) > \bar{u}$ ，其中 $\bar{u}$ 为平均紧迫性，因此 $\mathbb{E}[a_i(t+1)] > \mu_i$ 。具体而言：

$$\mathbb{E}[(a_i(t+1) - \mu_i) \cdot e_i(t)] \leq -\eta \cdot e_i(t)^2 \quad (12)$$

其中 $\eta = \frac{C}{N \cdot (deadline_{max} - t)^2} > 0$ 为恢复性漂移系数。该界成立，是因为紧迫性随剩余工作量单调增加，分配又与紧迫性成比例。显式推导使用紧迫性在公平分配点附近的 Taylor 展开。

利用这一恢复性漂移性质，我们对逐周期漂移给出界限：

$$\mathbb{E}[\Delta V(t) | V(t)] \leq -2\eta \cdot V(t) + N \cdot B^2 \quad (13)$$

其中 $B = \max_i |a_i(t) - \mu_i|$ 限定每周期的最大偏差。

集中分析的设置. 定义 $Z(t) = V(t) + NB^2/(2\eta)$ 。当 η 与零保持有界距离 ($deadline_{max} - t$ 有界时保证) 时， $Z(t)$ 非负，并满足下面的界。

集中性. 应用漂移加抖动界的最大不等式形式 [17]：

$$Pr \left[\max_{t \leq n} V(t) \geq \lambda^2 \right] \leq \frac{\mathbb{E}[V(0)] + NB^2 n / (2\eta)}{\lambda^2} \quad (14)$$

取 $\lambda = \epsilon \cdot \mu_i \cdot n$ ，得到所述界。 □

多租户干扰下实测 99.2% 的 SLO 达标率 (§9.6, 表 6)，与该界方向一致。定理 2 形式上覆盖满足 $W_i \leq C/N$ 的租户；重租户不满足这一假设，但经验上仍达到相近 SLO (99.1%，表 6)。

7 理论分析

本节提供理论依据，解释为何工作流感知调度相对请求级方法具有根本优势，并刻画驱逐策略的最优程度。

7.1 缓存效率分析

我们分析请求级与工作流感知调度器之间的缓存效率差距，给出启发性观察以及形式竞争比界。

观察 1 (请求级缓存的低效). 考虑具有 k 个顺序 LLM 推理步骤的智能体任务，每步产生 c 个词元的 KV 缓存，步骤间穿插工具调用。不维护会话状态的请求级调度器必须独立路由请求，可能路由到未持有该会话缓存状态的工作节点。内存压力下，这类调度器可能在工具调用空闲期驱逐缓存。最坏情况（每次工具调用后完全驱逐）下，总重新生成成本为 $\sum_{j=1}^k j \cdot c = O(k^2 \cdot c)$ 个词元。具有完美预测的工作流感知调度器达到 $O(c)$ 的重新生成成本（仅初始预填充）。

这一观察解释了工作流感知为何有益，却未刻画在线调度器能达到的界。因此我们给出形式竞争比分析。

定义 3 (KV 缓存驱逐的竞争比). 对于在线驱逐策略 $\mathcal{A}$ 和工作负载 σ ，令 $Cost_{\mathcal{A}}(\sigma)$ 为总缓存重新生成成本（预填充词元数）。竞争比为：

$$CR(\mathcal{A}) = \sup_{\sigma} \frac{Cost_{\mathcal{A}}(\sigma)}{Cost_{OPT}(\sigma)} \quad (15)$$

其中 $Cost_{OPT}(\sigma)$ 为完全知晓未来的 Bélády 最优离线策略 [4] 达到的成本。

近期理论工作 [63] 表明，在对抗性场景下，前缀树中基于 LRU 的驱逐可能退化为 $O(n)$ 竞争比，而随机算法达到 $O(\log n)$ 。其他理论基础包括完全对抗性在线调度中无法实现常数竞争比的结果 [29]，以及多步智能体网络中非空闲策略的分析 [37]。我们的 WA-LRU 利用工作流结构，达到良好的经验竞争比：

定理 3 (WA-LRU 竞争比界). 假设 AEG 预测以 $1 - \epsilon$ 的概率正确，工具调用时长遵循方差有界的经验分布，WA-LRU 达到如下经验竞争比：

$$CR_{\text{empirical}}(\text{WA-LRU}) \leq 1 + \epsilon \cdot k_{\max} + O\left(\frac{\sigma_{\text{tool}}}{TTL_{\text{adaptive}}}\right) \quad (16)$$

其中 $k_{\max}$ 为最大任务长度， σ_{tool} 为工具延迟标准差， TTL_{adaptive} 为自适应 TTL 设置。

证明概要. WA-LRU 仅在以下情况产生重新生成成本：(1) AEG 误预测（步骤中的 ϵ 比例），每次最多花费 $k_{\max} \cdot c$ 个词元；(2) 对长尾工具调用低估 TTL（其比例受 $O(\sigma_{\text{tool}}/TTL_{\text{adaptive}})$ 限定）。预测与 TTL 正确时，缓存跨全部步骤保留，与 OPT 一致。 $\square$

上述界以分布假设为条件（预测误差 ϵ 有界、工具延迟方差有界），因此是在这些假设下的期望情形竞争比，并非最坏情况的对抗性界。KV 缓存轨迹上在线策略的最坏情况竞争比仍是开放问题；近期工作 [63, 29] 为密切相关的在线问题建立了下界。

经验验证. 表 2 展示通过同时用 WA-LRU 与 Bélády 预知未来策略重放生产轨迹得到的经验竞争比。WA-LRU 在 SWE-bench 上达到 $1.31 \times$ （预测误差 $\epsilon = 0.13$ ， $k_{\max} = 150$ ，由此得到的最坏情况界 $1 + \epsilon \cdot k_{\max} \approx 20.5$ 在经验工作负载上远不紧，此时占主导的是平均任务长度 $k_{\text{avg}} = 37$ ），明显优于 LRU ($2.84 \times$) 与前缀缓存 ($1.86 \times$)。这些结果验证，工作流感知在现实智能体工作负载上接近最优效率。

表 2: 生产轨迹上, 相对 Bélády 最优离线算法的各驱逐策略竞争比。越低越好 (1.0 = 最优)。

策略	SWE-bench	WebArena	均值
标准 LRU	2.84	2.12	2.48
LRU + 前缀 (vLLM v0.5)	1.97	1.74	1.86
WA-LRU (本文)	1.31	1.28	1.30

7.2 WA-LRU 相对 Bélády 的竞争比: 经验结果

8 实现

我们将 SAGA 实现为 vLLM v0.6.0 [34] (V1 引擎) 的扩展, 约包括 8.5K 行 Python 与 1.2K 行 C++/CUDA, 组织为四个组件: 工作流分析器解析智能体框架注释 (LangChain 回调 [35]、AutoGen 消息日志 [64]) 构建 AEG, 对未注释框架回退为模式推断 (§3.3); 分布式调度器基于 Ray [44] 和 gRPC 构建 (工作节点至协调器的 P99 延迟 <5ms), 实现全局协调器与本地调度器扩展; KV 缓存管理器扩展 vLLM 的 PagedAttention [34], 提供 WA-LRU 驱逐、TTL 跟踪和独立 CUDA 流上的推测式预取 [50], 以便与解码核重叠; 公平性模块实现 AFS 计算 (§6.1) 和优先级驱动的容量分配。SAGA 作为独立服务运行, 拦截智能体框架请求并路由到 vLLM 工作节点; 无需改动智能体代码, 可选框架注释能够提高工作流推断准确性。

9 评估

我们从五个维度评估 SAGA: (1) 端到端性能; (2) 各组件有效性; (3) 多租户公平性; (4) 系统开销; (5) 对参数和设计选择的敏感性。

9.1 实验设置

硬件。 8 个节点, 每个配备 8 张 NVIDIA A100-80GB GPU (HBM2e, 2TB/s 带宽)、2 颗 AMD EPYC 7763 CPU (总计 128 核)、1TB DDR4-3200 内存及 4×3.84TB NVMe SSD。节点通过支持 GPUDirect RDMA 的 200Gbps InfiniBand HDR 互联 [45]。总计 64 GPU、1024 CPU 核和 5.12TB GPU 内存。

软件。 Ubuntu 22.04、CUDA 12.1.1 (驱动 530.30.02)、Python 3.10.12、PyTorch 2.1.2+cu121、vLLM 0.6.0、FlashAttention 2.5.6 [11]、Ray 2.9.0。模型为 Llama-3-70B-Instruct [43], 每个实例跨 4 GPU 进行张量并行。

工作负载。

- **SWE-bench** [31]: 500 个 “verified” 子集任务 (原作者为保证可处理性而选择), 具有智能体轨迹 (平均 37 步, 最多 150 步)。每步包含 2–4K 提示词元、100–500 输出词元。
- **WebArena** [72]: 完整 812 个浏览器任务 (平均 18 步)。每步包含 4–8K 提示词元 (包括页面内容)、50–200 输出词元。
- **由 BurstGPT 衍生** [62]: 具有 10 个租户的合成多租户工作负载, 分为 3 个 “重” 租户 (持续运行 100 步智能体)、4 个 “中” 租户 (间歇运行 30 步智能体), 及 3 个 “轻” 租户 (偶尔运行 10 步智能体)。每个租户按泊松过程到达任务, 重/中/轻租户平均速率约为每租户每分钟 16 / 8 / 4 个任务; 选择这些速率, 将集群总提交负载推至峰值吞吐量的约 80%, 进入用于检验 SAGA 公平性机制的竞争区间。请求结构和提示词元分布从 BurstGPT 轨迹采样; 到达时间由上述泊松过程决定 (没有使用 BurstGPT 原始到达时间戳, 因为轨迹仅包含单租户)。SWE-bench 和 WebArena 是任务定义, 并非到达轨迹; 对于 §9.2 的端到端测量, 我们按相同泊松计划重放它们 (单租户, $\lambda \approx 8$ 个任务/分钟)。

基线。

- **vLLM [34]**: v0.6.0 (V1 引擎), PagedAttention 配合 FCFS 调度。
- **vLLM+APC**: vLLM v0.15.1, 启用自动前缀缓存和 PrefixCacheAffinityRouter (`--enable-prefix-caching --enable-affinity-routing`)。这代表当时先进的 vLLM 配置, 同时支持前缀复用和基于亲和性的路由。注意: vLLM 的亲和路由器运行在前缀层面, 并非会话层面, 也不跨工具调用边界保留会话专属 KV 缓存。
- **SGLang [70]**: v0.5.8, 具有 RadixAttention、零开销批调度器及缓存感知负载均衡。
- **Llumnix [59]**: v1.2, vLLM 配合实时迁移进行负载均衡。
- **TRT-LLM+Scaffolding [46]**: TensorRT-LLM v1.1, 配合用于多步推理的 Scaffolding 框架及 KV Cache Connector API。
- **vLLM+KVFlow**: 我们在 vLLM v0.6.0 上重新实现的 KVFlow [48]。与原论文核验后, 达到其基准配置报告吞吐量的 96% (4% 差距归因于缓存策略粒度的实现差异)。

指标。

- **任务完成时间 (TCT)**: 从提交到获得结果的端到端时间 (秒)。
- **GPU 内存利用率**: 存放有用 KV 缓存的 GPU 内存比例。
- **吞吐量**: 每分钟完成的任务数。
- **SLO 达标率**: 在截止时间 (预期时间的 $1.5\times$) 内完成的任务比例。

方法。全部实验使用不同随机种子重复 10 次, 报告均值 $\pm$ 标准差。采用双侧 Welch t 检验评估统计显著性; * 表示 $p < 0.05$, ** 表示 $p < 0.01$, *** 表示 $p < 0.001$ 。排除三次预热运行, 移除超过 $1.5\times$ IQR 的离群值 (占测量 $<2\%$)。

9.1.1 基线版本时效性讨论

主要实现扩展 vLLM v0.6.0, 并与各主要系统的最新发行版比较。**关键比较**: 启用自动前缀缓存 (APC) 和 PrefixCacheAffinityRouter 的 vLLM v0.15.1 代表当时先进配置。该配置支持前缀共享, 将相似前缀请求路由到相同工作节点。如表 3 所示, vLLM+APC 相对早期 vLLM 版本有明显改善, 但 SAGA 仍达到 $1.73\times$ 加速 ($p < 0.001$), 原因如下:

1. **会话亲和性与前缀亲和性**: vLLM 的亲和路由器按共享前缀 (系统提示、工具定义) 分组请求, 却不跟踪会话身份。具有相同前缀、但不同对话历史的智能体会话无法区分。SAGA 按会话 ID 路由, 确保一个任务的全部步骤到达相同工作节点。
2. **工具调用 TTL**: vLLM 缓存在空闲期使用标准 LRU 驱逐。较长工具调用 (中位数 1.2s, P99 45s) 期间, 会话 KV 缓存可能因内存压力被驱逐。SAGA 的工作流感知 TTL 预测工具完成时间, 并据此保留缓存。
3. **任务级公平性**: vLLM 调度优化每个请求的延迟。多租户负载下, 轻租户可能饥饿。SAGA 的 AFS 调度在任务层面提供完成时间公平性。

与 TensorRT-LLM Scaffolding 比较: TRT-LLM v1.1 的 Scaffolding 框架通过 KV Cache Connector API 支持多步推理, 但关注单节点推理时计算, 而非分布式集群调度。SAGA 相对 TRT-LLM+Scaffolding 的 $1.60\times$ 优势 (表 3), 来自集群范围的会话亲和性与工作窃取。

模型规模讨论。评估使用采用 GQA 的 Llama-3-70B-Instruct ($n_{kv} = 8$), 每个 32K 会话产生 ~ 10.7 GB KV 缓存。SAGA 收益随模型规模扩大而增长, 因为 KV 缓存重新生成成本与模型维度成比例: 较小模型 (8B, ~ 1.5 GB 缓存) 每步重生成约需 ~ 0.3 s, 收益适中; 较大模型 (405B, 跨 TP 组 ~ 50 GB 以上缓存) 每步重生成约需 ~ 5 s, 带来成比例的更大收益。MoE 架构需要感知路由的扩展 (§1.5 已说明)。

9.1.2 作为替代架构的 CPU 换出

HBM 保留的自然替代方案，是在工具调用间隙经 PCIe 将空闲 KV 缓存卸载到主机 DRAM。我们出于三个量化原因选择结合预测式驱逐的 HBM 保留；将两种架构视为互补，而非竞争。

(1) **PCIe 往返时间主导短工具延迟。**对 10.7 GB 缓存 (Llama-3-70B、32K 上下文、GQA $n_{kv} = 8$)，采用 A100 服务器典型的 25 GB/s 实际持续带宽 [7]，经 PCIe Gen4 $\times 16$ 单向传输需要 ≈ 430 ms，因此不受竞争影响的换出加换入往返为 ≈ 860 ms。表 1 四类工具中三类（文件操作 P50=45 ms、代码执行 P50=180 ms、数据库查询 P50=120 ms）完成快于这一往返，使换出对最常见请求成为纯开销。

(2) **多租户 PCIe 竞争使该界恶化。**由 BurstGPT 衍生的工作负载 (§9.6) 下，PCIe 与模型权重加载、主机到设备张量复制及 NCCL 阶段共享，持续带宽下降至峰值的 50% 以下，使往返加倍至 ≈ 1.7 s，并将收支平衡点推到全部工具类型 P95 之后。

(3) **SAGA 的内存区间不需要换出。**表 3 显示，SAGA 内存利用率为 71 至 75%，高于 vLLM+APC 的 59 至 61%，但仍预留 25 至 29% HBM。预测式 WA-LRU (§4.1) 与按压力缩放的 TTL (式 6) 使该区间无需换出。在超额订阅区间 ($>95\%$ 利用率，超出评估范围；§1.5)，换出仍是互补方案；FlexGen [55] 广泛探索了主机 DRAM 卸载，将其集成为 WA-LRU 预测下的第三驱逐层，是直接可行的未来工作。

9.2 端到端性能

表 3 展示智能体基准的端到端性能，并给出完整统计细节。

在 SWE-bench 上，SAGA 相对 vLLM v0.6.0 达到 $3.01 \times \pm 0.16$ 加速 ($p < 0.001$)。相对先进的 vLLM+APC 基线 (v0.15.1, 启用自动前缀缓存及亲和路由)，SAGA 仍达到 $1.73 \times$ 加速 ($p < 0.001$)，确认工作流级优化提供了仅靠前缀缓存和亲和路由无法实现的额外收益。相对 KVFlow 的 $1.47 \times$ 改善说明，SAGA 的集成方案（分布式调度 + TTL 策略 + AFS 公平性）优于单独的工作流感知缓存。

分解分析。图 1(a) 展示 SWE-bench 任务时间分解。vLLM v0.6.0 将时间的 38% 用于工具调用后的 KV 缓存重新生成。vLLM+APC 通过前缀共享及亲和路由将其降至 22%，但较长工具调用期间仍会驱逐会话专属缓存。SAGA 通过工作流感知 TTL 将重新生成降至 8%。

9.3 消融研究

表 4 通过 SWE-bench 消融实验量化各组件贡献。

会话亲和性提供最大收益（禁用后减慢 96%），因为它将相关请求路由到同一工作节点，直接避免缓存重新生成。工作流感知驱逐和 TTL 共同贡献 42 至 54% 改善。推测式预取通过使缓存加载与工具执行重叠，提供 19% 改善。AFS 在单基准场景中贡献较小的 8% 改善，但在多租户竞争下变得必不可少 (§9.6)。

9.4 模式推断评估

表 5 比较有无框架提示时的性能。

模式推断预测工作流结构的准确率为 87%，相对显式提示性能下降 15.6%，但相对 vLLM v0.6.0 仍提供 $2.60 \times$ 加速 (表 3)。13% 错误率主要表现为分支点的后继误预测（例如智能体进入新步骤时，却预测为“重试”循环），导致为错误分支不必要地保留缓存。这些错误不会级联：实际下一条请求到达时，系统检测误预测，并纠正后续步骤的路由。

9.5 可扩展性与负载均衡

SAGA 在固定工作负载下从 8 扩展到 64 GPU，达到 $6.4 \times$ 加速 (80% 效率)；至 512 个并发智能体时，弱扩展接近线性（每次翻倍 $0.94 \times$ ）。在 32 GPU 子集上（缩小规模以将执行模型影响与扩

表 3: 智能体基准上的端到端性能。TCT = 任务完成时间 (秒); Mem = GPU 内存利用率 (%)。数值为 10 次试验的均值 $\pm$ 标准差。相对每个基线的显著性: *** $p < 0.001$, ** $p < 0.01$ (成对 Welch t 检验)。

系统	SWE-bench		WebArena	
	TCT (s)	内存%	TCT (s)	内存%
vLLM v0.6.0	612.3 $\pm$ 32.1	42.1 $\pm$ 2.3	178.4 $\pm$ 14.2	45.3 $\pm$ 2.1
vLLM+APC v0.15.1	352.1 $\pm$ 21.4	58.7 $\pm$ 2.8	127.3 $\pm$ 10.1	61.2 $\pm$ 2.5
SGLang v0.5.8	387.2 $\pm$ 24.3	56.2 $\pm$ 2.6	138.7 $\pm$ 11.3	58.9 $\pm$ 2.4
Llumnix v1.2	498.1 $\pm$ 28.7	48.3 $\pm$ 2.4	156.2 $\pm$ 12.8	51.7 $\pm$ 2.2
TRT-LLM+Scaff.	324.6 $\pm$ 19.8	61.4 $\pm$ 2.7	118.9 $\pm$ 9.4	63.8 $\pm$ 2.6
vLLM+KVFlow	298.4 $\pm$ 18.2	64.1 $\pm$ 2.9	108.2 $\pm$ 8.7	66.4 $\pm$ 2.7
SAGA	203.4$\pm$12.8	71.3$\pm$2.4	82.1$\pm$6.8	74.6$\pm$2.3

SAGA 相对基线的加速比 (TCT 比值):

相对 vLLM v0.6.0	3.01 $\times$ ***	2.17 $\times$ ***
相对 vLLM+APC	1.73 $\times$ ***	1.55 $\times$ ***
相对 SGLang	1.90 $\times$ ***	1.69 $\times$ ***
相对 Llumnix	2.45 $\times$ ***	1.90 $\times$ ***
相对 TRT-LLM	1.60 $\times$ ***	1.45 $\times$ **
相对 KVFlow	1.47 $\times$ ***	1.32 $\times$ **

相对 *vLLM+APC* 的几何平均加速比: **1.64 $\times$**

表 4: SWE-bench 上的消融研究。每行从完整系统移除一个组件。数值为 10 次试验的均值 $\pm$ 标准差。

配置	TCT (s)	相对完整系统
完整 SAGA	203.4 $\pm$ 12.8	—
无工作流感知驱逐	312.8 $\pm$ 18.3	+54%
无工具调用 TTL	289.1 $\pm$ 16.7	+42%
无推测式预取	241.6 $\pm$ 14.2	+19%
无会话亲和性	398.2 $\pm$ 22.4	+96%
无工作窃取	267.3 $\pm$ 15.9	+31%
无 AFS 公平性	218.7 $\pm$ 13.1	+8%

展影响分离), 工作节点利用率范围由无工作窃取的 23 至 94%, 收窄为启用窃取的 68 至 79%; 迁移开销均值 230 ms、P95 890 ms, 每个任务平均发生 2.3 次。

表 5: 性能比较: 框架提示与模式推断。准确率衡量保留测试轨迹中正确预测下一步节点转移的比例。

模式	TCT (s)	AEG 准确率	相对提示
有提示	203.4 $\pm$ 12.8	100%	—
模式推断	235.2 $\pm$ 14.6	87%	+15.6%
无 AEG (基线)	398.2 $\pm$ 22.4	—	+95.8%

表 6: 按租户类型统计的 SLO 达标率 (截止时间内完成任务的百分比)。

系统	重租户	中租户	轻租户	总体
vLLM	89.4	72.1	43.2	67.3
SGLang	91.2	78.6	51.4	73.4
Llumnix	92.8	81.3	58.9	77.2
SAGA	99.1	99.4	98.7	99.2

表 7: SAGA 开销分解 (64 GPU, 32 租户)。

组件	均值 (ms)	P95 (ms)	CPU (%)
协调器周期	12.3	28.7	4.2
AFS 计算	3.1	8.4	—
AEG 构建	45.2	112.8	—
工作窃取 (迁移)	230	890	—

9.6 多租户公平性

我们使用由 BurstGPT 衍生、包含 10 个不同负载强度租户的工作负载, 评估多租户行为。

SLO 达标率。表 6 展示 SLO 达标率 (任务在预期时间的 $1.5\times$ 内完成)。SAGA 总体达标率为 99.2%, vLLM 为 67.3%。轻租户改善最明显 (98.7% 相对 43.2%), 验证定理 2。

公平性分析。vLLM 对轻租户表现出高方差与长尾 ($P99 = \text{预期 TCT 的 } 12.4\times$)。SAGA 在全部租户类型间提供一致完成时间 ($P99 < \text{预期的 } 1.8\times$)。

9.7 系统开销分析

表 7 分解 SAGA 的调度开销。

协调器总 CPU 开销为 4.2%, 为应用工作负载留下 95.8%。AFS 计算随租户数量线性增长, 但仍可忽略 (32 租户耗时 3.1ms)。

9.8 执行策略权衡

表 8 在 SWE-bench、32 GPU 上比较不同执行策略 (缩小规模以将策略影响与集群规模影响分离)。BFS 与 DFS 的权衡是并行系统中已有充分研究的现象 [24], 在智能体调度中表现明显。

纯 BFS 最大化吞吐量 (12.4 个任务/分钟), 但驱逐率达 78%。SAGA 的混合方案以低 30% 的吞吐量达到最优 TCT (203.4s), 适合对延迟敏感交互部署 [15, 72]。

表 8: SWE-bench 上的执行策略比较 (32 GPU)。

策略	TCT (s)	吞吐量	驱逐率
纯 BFS	487.2 $\pm$ 28.4	12.4 任务/分钟	78%
纯 DFS	623.1 $\pm$ 34.2	4.2 任务/分钟	3%
混合 (SAGA)	203.4 $\pm$ 12.8	8.7 任务/分钟	12%

表 9: SWE-bench 上的参数敏感性分析。TCT $_{\Delta}$ 为测试范围内相对默认值的最大变化。

参数	默认值	范围	TCT $_{\Delta}$	依据
α (最近访问权重)	0.3	[0.2, 0.4]	<5%	敏感性分析
β (复用权重)	0.5	[0.4, 0.6]	<8%	敏感性分析
γ (尺寸权重)	0.2	[0.1, 0.3]	<3%	尺寸作为同分决胜因素
θ (路由)	0.8	[0.6, 0.95]	<5%	20% 余量 [59]
$threshold_{low}$	0.7	[0.6, 0.8]	<4%	软压力起点 [34]
$threshold_{high}$	0.9	[0.85, 0.95]	<6%	强制驱逐限制
T_{idle} (窃取触发)	100ms	[50, 200]ms	<7%	摊销窃取成本
R_{max} (负载比)	2.0	[1.5, 3.0]	<4%	不平衡容忍度
TTL_{max}	300s	[120, 600]s	<3%	P99 工具延迟上限
θ_{conf} (AEG)	0.7	[0.5, 0.9]	<6%	精确率与召回率权衡

9.9 参数敏感性

表 9 总结全部可配置参数的敏感性分析。

表 9 的测试范围相对各默认值跨越 $\pm 33\%$ 至 $\pm 50\%$ ；没有单参数扰动导致 $>8\%$ TCT 变化。这种稳健性来自结构，而非调参运气：按表 4 消融，会话亲和性贡献最大（移除后 TCT 增加 96%，由 203.4s 增至 398.2s），而会话亲和性是二元的：会话要么到达其缓存所在工作节点，要么没有到达。其余连续参数仅以平滑加权项进入归一化优先级中的驱逐分数（式 1）与 TTL 公式（算法 1）。多维对抗性扰动（例如同时将 α, β, γ 设为边界值）尚未经验刻画，留待未来研究；我们预计这种区间超出合理部署会选择的范围。最敏感的单个参数为 β （复用权重）与 T_{idle} （窃取触发），反映它们对缓存保留与负载均衡的直接影响。因此，我们将 SAGA 描述为单维稳健：在已测试扰动下，采用近似默认值的部署相对调优运行，TCT 恶化最多约 $\sim 8\%$ 。

9.10 对工具延迟方差的敏感性

表 10 展示 SAGA 性能随工具延迟方差的变化；方差变化以工具调用时长的变异系数（CV）衡量。我们在固定平均工具延迟的同时，合成改变 CV。

SAGA 的自适应 TTL 在 CV=2.0 以内保持一致性能（TCT 恶化 24%）。超过 CV=2.0 后，极端离群值引发过早驱逐，使 TTL 预测准确性明显下降。实际生产工具延迟的 CV ≈ 1.0 至 1.5（表 1），处于 SAGA 有效范围之内。

10 相关工作

表 11 将 SAGA 与直接相关的程序感知系统比较。

表 10: 对工具延迟方差（变异系数）的敏感性。平均工具延迟固定为 1.2s。

CV	TCT (s)	TTL 准确率	驱逐率	相对 CV=0.5
0.5	195.1±11.2	96%	9%	—
1.0	203.4±12.8	93%	12%	+4%
1.5	218.6±15.3	88%	18%	+12%
2.0	241.3±18.7	82%	24%	+24%
3.0	298.4±24.1	71%	35%	+53%

表 11: 与直接相关的程序感知 LLM 服务系统比较。

系统	发表场合	分布式 调度	工具 TTL 策略	公平性 保证	竞争 比
Parrot [38]	OSDI'24	×	×	×	×
Autellix [40]	arXiv'25	✓	×	×	×
Pie [23]	SOSP'25	×	✓	×	×
KVFlow [48]	NeurIPS'25	×	✓	×	×
SAGA	本文	✓	✓	✓	✓

LLM 服务。 vLLM [34]、SGLang [70]、Orca [68] 和 TensorRT-LLM [46] 优化请求级指标；SAGA 增加工作流级优化。

感知程序的服务与分布式推理。 Parrot [38] 引入语义变量，但缺少分布式调度与公平性；Autellix [40] 提出程序级公平性；Pie [23] 通过 inferlet 分解生成；KVFlow [48] 和 Continuum [30] 引入工作流感知驱逐；Llumnix [59] 支持实时 KV 迁移，SOLA [26] 优化 SLO 达标率，但二者均不以工作流级调度为目标。SAGA 的独特定位（表 11），是以量化工作流感知在线缓存管理极限的经验竞争比界为基础，统一这些维度。

公平性与缓存。 DRF [21]、VTC [56] 和 Themis [41] 处理资源公平性；SAGA 将其扩展至任务完成公平性。WA-LRU 相对 Bélády 最优策略 [4] 达到 $1.31\times$ 竞争比。

11 结论与未来工作

我们提出 SAGA，一种面向多步 AI 智能体工作负载、将智能体程序作为一等可调度单位的分布式调度器。将三个经典系统原则（工作流调度、知情缓存、应用级公平性）适配到复合 AI 领域后，SAGA 在 SWE-bench 和 WebArena 上相对启用自动前缀缓存的 vLLM v0.15.1，分别将任务完成时间改善 $1.73 \times \pm 0.11$ 和 $1.55 \times \pm 0.09$ （几何平均 $1.64\times$ ， $p < 0.001$ ），将内存利用率提高 $1.22 \times \pm 0.05$ ，并在多租户干扰下达到 99.2% SLO 达标率。相对不感知工作流的系统，改善最高达到 $3.01\times$ 。这些收益伴随相对吞吐量最优批调度约 $\sim 30\%$ 的吞吐量下降 (§9.8, 表 8)；适合对延迟敏感交互部署，但不适合批处理。

技术贡献与定位。 我们形式化智能体执行图，并表明 WA-LRU 达到 Bélády 最优离线策略的 $1.31\times$ 以内。据我们所知，这是工作流感知 KV 缓存管理的首个经验竞争比分析。AFS 的 Lyapunov 漂移分析提供形式完成时间界，并明确推导恢复性漂移性质。近期感知程序的服务系统（Parrot、Autellix、Pie、KVFlow）各自解决一到两个问题维度；SAGA 的作用，是在经验竞争比界之下结

合工作流感知缓存、分布式调度、工具感知 TTL 与任务级公平性，首次量化工作流 DAG 可观察后在线调度器能够多接近离线最优缓存管理。表 11 详述各维度的区别。

开放研究方向。本工作打开以下方向：

1. **最优 TTL 预测：**学习具有可证明遗憾界的 TTL 策略，类似于具有部分反馈的在线学习 [36]。
2. **更紧竞争比：**相对 Bélády 实测 $1.31\times$ 表明仍有改进空间；使用 AEG 预测可以达到的信息论下界是什么？
3. **复杂度：**具有缓存约束的最优工作流感知调度是否 NP 困难？形式结果能够为启发式方法提供依据。
4. **地理分布调度：**将 AFS 扩展到跨数据中心的智能体，并考虑依赖网络的迁移成本。
5. **多智能体协调：**联合优化互相交互的智能体执行图（协作编码、协商）。
6. **集成推测：**将推测执行 [67, 52] 与工作流感知调度结合，获取乘性收益。

图内文字中文对照与译注

图 1 的图内说明。 (a) Time breakdown = 时间分解；(b) GPU memory utilization = GPU 内存利用率；(c) Latency overhead = 延迟开销；ideal = 理想值； $4.0\times$ lower = 低 4.0 倍。Inference = 推理；KV regeneration = KV 缓存重新生成；Tool wait = 工具等待；Other = 其他。pp 表示百分点；APC 表示自动前缀缓存；vLLM 与 SAGA 保留系统名称。

图 2 的图内说明。 1 Agent Interface = 第 1 层：智能体接口；T/A/O 分别表示 Thought/Action/Observation，即思考/行动/观察；Agent Execution Graph from LangChain · AutoGen · CrewAI = 来自这些框架的智能体执行图；AEG submission = AEG 提交。2 Global Scheduler = 第 2 层：全局调度器；Affinity Router = 亲和路由器；cache locality = 缓存局部性；AFS Engine = AFS 引擎；agent-level fairness = 智能体级公平性；Work Stealer = 工作窃取器；load balance = 负载均衡；Cluster State = 集群状态；worker loads · KV-cache map · fairness counters = 工作节点负载、KV 缓存映射、公平性计数器；workflow-atomic dispatch = 工作流原子派发。3 Worker Pool = 第 3 层：工作节点池；vLLM + extensions = vLLM 加扩展；Worker 1/2/p = 工作节点 1/2/p；KV cache slots = KV 缓存槽位；WA-LRU = 工作流感知 LRU；KV cache legend = KV 缓存图例；session $\alpha/\beta/\gamma$ = 会话 $\alpha/\beta/\gamma$ ；free = 空闲。

图 3 的图内说明。 terminal = 终止；KV cache: 12K tokens preserved = KV 缓存：保留 12K 词元；read file = 读取文件；edit code = 编辑代码；run test = 运行测试；bar width $\propto \sqrt{\text{idle time}}$ = 条形宽度与空闲时间平方根成比例； $53\times$ variance motivates TTL prediction = 53 倍差异促成 TTL 预测。图中的 ms 为毫秒，s 为秒；节点和边上数值保持原样。

公式内英文说明。 式 2 的 normalized recency = 归一化的最近访问间隔；式 3 的 normalized size = 归一化尺寸。式 7 中 if/and/otherwise 分别表示“若/且/否则”；函数名、变量名和数学环境保持原样。

模型与实测界的区别。 AEG 的定义允许一般有向依赖，图 3 明确绘有重试后向边；因此即使正文个别位置使用“工作流 DAG”，所示模型也不能一概视为严格无环图。 $1.31\times$ 为 SWE-bench 轨迹上的经验成本比；定理 3 给出的依赖预测误差、任务长度和工具延迟的条件界，并未证明所有轨迹都满足 1.31。原文也明确区分期望情形与对抗性最坏情形，并报告代入参数后约为 20.5 的不紧界。

原文中需独立核验的推导。 本译文完整保留定理 2 及其证明概要。概要声称服务不足即可推出高于平均的紧迫性，但未写出在不同工作量与不同截止时间下使该推论成立的完整条件；最后展示的概率上界随 λ^{-2} 衰减，未展示如何进一步得到所述带 $\log(N/\delta)$ 的偏差阶。因此，这些内容应视为作者提出的理论主张与证明概要，独立使用该定理前仍需补全推导；实测 SLO 达标率本身不能替代证明。

公式与描述中的其他差异。 重叠函数的形式定义以当前缓存词元数为分母，而随后线性 ReAct 的估计式以当前词元加预计观察词元数为分母，二者并非同一归一化量。式 6 仅以零为下界裁剪，利用率超过高阈值时可能大于 1，而算法输入写作 $m \in [0, 1]$ ；正文未展示额外的上界裁剪。工作窃取触发段先写两个条件以“或”连接，防抖段又要求同时满足；均按原文保留。CPU 换出讨论称 1.7 秒超过所有工具类 P95，但表 1 的代码执行与网页调用 P95 分别为 2.4、4.5 秒。执行策略表（32 GPU）与端到端表（64 GPU）均列出 SAGA 的 203.4 秒，且软件段将 vLLM 0.6.0 标为 V1 引擎；这些表述和数据保留原样，未替作者更改。背景对循环“第 3 / 6 行”的引用在所供源码中没有对应的循环代码清单。

参考文献

- [1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with sarathi-serve. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 117–134. USENIX Association, 2024.
- [2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 4895–4901. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.EMNLP-MAIN.298.
- [3] Amazon Web Services. Amazon Q Developer. AWS product page, 2024. URL <https://aws.amazon.com/q/developer/>.
- [4] Laszlo A. Belady. A study of replacement algorithms for virtual-storage computer. *IBM Syst. J.*, 5(2):78–101, 1966. doi: 10.1147/SJ.52.0078.
- [5] Robert D. Blumofe. Scheduling multithreaded computations by work stealing. In *35th Annual Symposium on Foundations of Computer Science, Santa Fe, New Mexico, USA, November 20-22, 1994*, pages 356–368. IEEE Computer Society, 1994. doi: 10.1109/SFCS.1994.365680.
- [6] Pei Cao, Edward W. Felten, Anna R. Karlin, and Kai Li. Implementation and performance of integrated application-controlled file caching, prefetching, and disk scheduling. *ACM Trans. Comput. Syst.*, 14(4):311–343, 1996. doi: 10.1145/235543.235544.
- [7] Esha Choukse, Pratyush Patel, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, Rodrigo Fonseca, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. *IEEE Micro*, 45(4):54–59, 2025. doi: 10.1109/MM.2025.3575361.
- [8] James C. Corbett, Jeffrey Dean, Michael Epstein, Andrew Fikes, Christopher Frost, J. J. Furman, Sanjay Ghemawat, Andrey Gubarev, Christopher Heiser, Peter Hochschild, Wilson C. Hsieh, Sebastian Kanthak, Eugene Kogan, Hongyi Li, Alexander Lloyd, Sergey Melnik, David

- Mwaura, David Nagle, Sean Quinlan, Rajesh Rao, Lindsay Rolig, Yasushi Saito, Michal Szymaniak, Christopher Taylor, Ruth Wang, and Dale Woodford. Spanner: Google’s globally distributed database. *ACM Trans. Comput. Syst.*, 31(3):8, 2013. doi: 10.1145/2491245.
- [9] Christian Corrò and Luca Chittaro. Exploring the potential and limitations of large language models to control the behavior of embodied persuasive agents. In *Persuasive Technology - 20th International Conference, PERSUASIVE 2025, Limassol, Cyprus, May 5-7, 2025, Proceedings*, Lecture Notes in Computer Science, pages 61–73. Springer, 2025. doi: 10.1007/978-3-031-94959-3_5.
- [10] CrewAI. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. GitHub repository, 2023. URL <https://github.com/crewAIInc/crewAI>.
- [11] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.
- [12] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.
- [13] Ewa Deelman, Karan Vahi, Gideon Juve, Mats Rynge, Scott Callaghan, Philip Maechling, Rajiv Mayani, Weiwei Chen, Rafael Ferreira da Silva, Miron Livny, and R. Kent Wenger. Pegasus, a workflow management system for science automation. *Future Gener. Comput. Syst.*, 46:17–35, 2015. doi: 10.1016/J.FUTURE.2014.10.008.
- [14] Peter J. Denning. Virtual memory. *ACM Computing Surveys*, 2(3):153–189, 1970. doi: 10.1145/356571.356573.
- [15] Thomas Dohmke. GitHub Copilot Workspace: Welcome to the Copilot-native developer environment. GitHub Blog, April 2024. URL <https://github.blog/news-insights/product-news/github-copilot-workspace/>.
- [16] Ulrich Drepper. What every programmer should know about memory. Whitepaper, Red Hat, Inc., November 2007. URL <https://people.freebsd.org/~lstewart/articles/cpumemory.pdf>.
- [17] Devdatt P. Dubhashi and Alessandro Panconesi. *Concentration of Measure for the Analysis of Randomized Algorithms*. Cambridge University Press, 2009. ISBN 978-0-521-88427-3.
- [18] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.*, 23:120:1–120:39, 2022.
- [19] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. Serverlessllm: Low-latency serverless inference for large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 135–153. USENIX Association, 2024.

-
- [20] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. Efficient LLM scheduling by learning to rank. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.
 - [21] Ali Ghodsi, Matei Zaharia, Benjamin Hindman, Andy Konwinski, Scott Shenker, and Ion Stoica. Dominant resource fairness: Fair allocation of multiple resource types. In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2011, Boston, MA, USA, March 30 - April 1, 2011*. USENIX Association, 2011.
 - [22] In Gim, Guojun Chen, Seung-Seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. In *Proceedings of the Seventh Annual Conference on Machine Learning and Systems, MLSys 2024, Santa Clara, CA, USA, May 13-16, 2024*. mlsys.org, 2024.
 - [23] In Gim, Zhiyao Ma, Seung-seob Lee, and Lin Zhong. Pie: A programmable serving system for emerging llm applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles, SOSP '25*, page 415–430, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400718700. doi: 10.1145/3731569.3764814. URL <https://doi.org/10.1145/3731569.3764814>.
 - [24] Goetz Graefe. Encapsulation of parallelism in the volcano query processing system. In *Proceedings of the 1990 ACM SIGMOD International Conference on Management of Data, Atlantic City, NJ, USA, May 23-25, 1990*, pages 102–111. ACM Press, 1990. doi: 10.1145/93597.98720.
 - [25] Maurice Herlihy. The art of multiprocessor programming. In *Proceedings of the Twenty-Fifth Annual ACM Symposium on Principles of Distributed Computing, PODC 2006, Denver, CO, USA, July 23-26, 2006*, pages 1–2. ACM, 2006. doi: 10.1145/1146381.1146382.
 - [26] Ke Hong, Xiuhong Li, Lufang Chen, Qiuli Mao, Guohao Dai, Xuefei Ning, Shengen Yan, Yun Liang, and Yu Wang. SOLA: optimizing SLO attainment for large language model serving with state-aware scheduling. In *Proceedings of the Eighth Conference on Machine Learning and Systems, MLSys 2025, Santa Clara, CA, USA, May 12-15, 2025*. OpenReview.net/mlsys.org, 2025.
 - [27] Cunchen Hu, Heyang Huang, Liangliang Xu, Xusheng Chen, Chenxi Wang, Jiang Xu, Shuang Chen, Hao Feng, Sa Wang, Yungang Bao, Ninghui Sun, and Yizhou Shan. Shuff-fleinfer: Disaggregate llm inference for mixed downstream workloads. *ACM Trans. Archit. Code Optim.*, 22(2), July 2025. ISSN 1544-3566. doi: 10.1145/3732941. URL <https://doi.org/10.1145/3732941>.
 - [28] InfiniBand Trade Association. InfiniBand architecture specification, volume 2, release 1.4. Industry standards specification, April 2020. URL <https://www.infinibandta.org/ibta-specification/>.
 - [29] Patrick Jaillet, Jiashuo Jiang, Konstantina Mellou, Marco Molinaro, Chara Podimata, and Zijie Zhou. Online scheduling for llm inference with kv cache constraints. *arXiv preprint*, arXiv:2502.07115, 2026. URL <https://arxiv.org/abs/2502.07115>.
 - [30] Matthijs Jansen, Linus Wagner, Animesh Trivedi, and Alexandru Iosup. Continuum: Automate infrastructure deployment and benchmarking in the compute continuum. In *Companion of the 2023 ACM/SPEC International Conference on Performance Engineering*,

- ICPE 2023, Coimbra, Portugal, April 15-19, 2023*, pages 181–188. ACM, 2023. doi: 10.1145/3578245.3584936.
- [31] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.
 - [32] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Shufan Liu, Xuanzhe Liu, and Xin Jin. Rag-cache: Efficient knowledge caching for retrieval-augmented generation. *ACM Trans. Comput. Syst.*, 44(1):2:1–2:27, 2026. doi: 10.1145/3768628.
 - [33] Sayash Kapoor, Benedikt Stroebel, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. AI agents that matter. *Trans. Mach. Learn. Res.*, 2025, 2025.
 - [34] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pages 611–626. ACM, 2023. doi: 10.1145/3600006.3613165.
 - [35] LangChain-AI. LangChain: Build context-aware reasoning applications. GitHub repository, 2022. URL <https://github.com/langchain-ai/langchain>.
 - [36] Tor Lattimore and Csaba Szepesvári. *Bandit Algorithms*. Cambridge University Press, July 2020. ISBN 9781108486828. doi: 10.1017/9781108571401.
 - [37] Yueying Li, Jim Dai, and Tianyi Peng. Throughput-optimal scheduling algorithms for llm inference and ai agents. *arXiv preprint*, arXiv.2504.07347, 2025. URL <https://arxiv.org/abs/2504.07347>.
 - [38] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. Parrot: Efficient serving of llm-based applications with semantic variable. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 929–945. USENIX Association, 2024.
 - [39] Yuhao Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference, ACM SIGCOMM 2024, Sydney, NSW, Australia, August 4-8, 2024*, pages 38–56. ACM, 2024. doi: 10.1145/3651890.3672274.
 - [40] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint*, arXiv.2502.13965, 2025. URL <https://arxiv.org/abs/2502.13965>.
 - [41] Kshiteej Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Shuchi Chawla. Themis: Fair and efficient GPU cluster scheduling. In *17th USENIX Symposium on Networked Systems Design and Implementation, NSDI 2020, Santa Clara, CA, USA, February 25-27, 2020*, pages 289–304. USENIX Association, 2020.

-
- [42] Nimrod Megiddo and Dharmendra S. Modha. ARC: A self-tuning, low overhead replacement cache. In *Proceedings of the FAST '03 Conference on File and Storage Technologies, March 31 - April 2, 2003, Cathedral Hill Hotel, San Francisco, California, USA*. USENIX, 2003.
 - [43] Meta. Llama 3 model card. Meta Llama documentation, 2024. URL https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.
 - [44] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In *13th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2018, Carlsbad, CA, USA, October 8-10, 2018*, pages 561–577. USENIX Association, 2018.
 - [45] NVIDIA Corporation. NVIDIA NVSwitch: The world’s highest-bandwidth on-node switch. NVIDIA Technical Overview, 2018. URL <https://images.nvidia.com/content/pdf/nvswitch-technical-overview.pdf>.
 - [46] NVIDIA Corporation. TensorRT-LLM: High-performance large language model inference. GitHub repository, 2023. URL <https://github.com/NVIDIA/TensorRT-LLM>.
 - [47] Kay Ousterhout, Patrick Wendell, Matei Zaharia, and Ion Stoica. Sparrow: distributed, low latency scheduling. In *ACM SIGOPS 24th Symposium on Operating Systems Principles, SOSP '13, Farmington, PA, USA, November 3-6, 2013*, pages 69–84. ACM, 2013. doi: 10.1145/2517349.2522716.
 - [48] Zaifeng Pan, AJJKUMAR PATEL, Yipeng Shen, Zhengding Hu, Yue Guan, Wan-Lu Li, Lianhui Qin, Yida Wang, and Yufei Ding. KVFlow: Efficient prefix caching for accelerating LLM-based multi-agent workflows. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=5Iw1nDtYmT>.
 - [49] R. Hugo Patterson, Garth A. Gibson, Eka Ginting, Daniel Stodolsky, and Jim Zelenka. Informed prefetching and caching. In *Proceedings of the Fifteenth ACM Symposium on Operating System Principles, SOSP 1995, Copper Mountain Resort, Colorado, USA, December 3-6, 1995*, pages 79–95. ACM, 1995. doi: 10.1145/224056.224064.
 - [50] Steve Rennich. CUDA C/C++ streams and concurrency. NVIDIA CUDA Training Webinar, 2012. URL <https://developer.download.nvidia.com/CUDA/training/StreamsAndConcurrencyWebinar.pdf>.
 - [51] Minsoo Rhu, Natalia Gimelshein, Jason Clemons, Arslan Zulfiqar, and Stephen W. Keckler. vdn: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *49th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2016, Taipei, Taiwan, October 15-19, 2016*, pages 18:1–18:13. IEEE Computer Society, 2016. doi: 10.1109/MICRO.2016.7783721.
 - [52] Yeonju Ro, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, Aditya Akella, Zhangyang Wang, Mattan Erez, and Esha Choukse. Sherlock: Reliable and efficient agentic workflow execution. *arXiv preprint*, arXiv.2511.00330, 2025. URL <https://arxiv.org/abs/2511.00330>.
 - [53] Matthew Rocklin. Dask: Parallel computation with blocked algorithms and task scheduling. In *Proceedings of the 14th Python in Science Conference, SciPy 2015, Austin, Texas, USA, July 6-12, 2015*, pages 126–132. scipy.org, 2015. doi: 10.25080/MAJORA-7B98E3ED-013.

-
- [54] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of LM agents with an lm-emulated sandbox. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.
 - [55] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. Flexgen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, Proceedings of Machine Learning Research, pages 31094–31116. PMLR, 2023.
 - [56] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. Fairness in serving large language models. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 965–988. USENIX Association, 2024.
 - [57] Daniel Dominic Sleator and Robert Endre Tarjan. Amortized efficiency of list update and paging rules. *Commun. ACM*, 28(2):202–208, 1985. doi: 10.1145/2786.2793.
 - [58] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. Dynamollm: Designing LLM inference clusters for performance and energy efficiency. In *IEEE International Symposium on High Performance Computer Architecture, HPCA 2025, Las Vegas, NV, USA, March 1-5, 2025*, pages 1348–1362. IEEE, 2025. doi: 10.1109/HPCA61900.2025.00102.
 - [59] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 173–191. USENIX Association, 2024.
 - [60] Douglas Thain, Todd Tannenbaum, and Miron Livny. Distributed computing in practice: the condor experience. *Concurr. Pract. Exp.*, 17(2-4):323–356, 2005. doi: 10.1002/CPE.938.
 - [61] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS’17*, page 6000–6010, Red Hook, NY, USA, 2017. Curran Associates Inc. ISBN 9781510860964. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
 - [62] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. Burstgpt: A real-world workload dataset to optimize LLM serving systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, V.2, KDD 2025, Toronto ON, Canada, August 3-7, 2025*, pages 5831–5841. ACM, 2025. doi: 10.1145/3711896.3737413.
 - [63] Fangzhou Wu, Sandeep Silwal, and Qiuyi Zhang. Randomization boosts KV caching, learning balances query load: A joint perspective. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=R7fv5NWfMm>.

-
- [64] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=BAakY1hNKS>.
 - [65] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.
 - [66] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
 - [67] Naimeng Ye, Arnav Ahuja, Georgios Liargkovas, Yunan Lu, Kostis Kaffes, and Tianyi Peng. Speculative actions: A lossless framework for faster AI agents. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=POG0k5wslg>.
 - [68] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*, pages 521–538. USENIX Association, 2022.
 - [69] Matei Zaharia, Omar Khattab, Lingjiao Chen, Jared Quincy Davis, Heather Miller, Christopher Potts, James Zou, Michael Carbin, Jonathan Frankle, Naveen Rao, and Ali Ghodsi. The shift from models to compound AI systems. Berkeley Artificial Intelligence Research (BAIR) Blog, February 2024. URL <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>.
 - [70] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. Sglang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.
 - [71] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2024, Santa Clara, CA, USA, July 10-12, 2024*, pages 193–210. USENIX Association, 2024.
 - [72] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.